ProtoGPU: A Semantic GPU ISA Prototyping Framework for Rapid Instruction Exploration

Han Zhenzhong
ProtoGPU Project
https://github.com/Han-Zhenzhong/ProtoGPU

Abstract

The evolution of GPU instruction sets is often constrained by long hardware design cycles and limited prototyping infrastructure. Evaluating new GPU instructions typically requires complex microarchitectural simulators or full hardware implementations, which significantly slows down the exploration of ISA extensions.

This paper presents **ProtoGPU**, a semantic GPU execution framework designed for rapid prototyping and validation of new GPU instructions. ProtoGPU executes CUDA and PTX programs using a functional SIMT execution model without requiring cycle-accurate hardware simulation. The framework introduces an **IR-based instruction expansion mechanism** that enables new instructions to be integrated by mapping them to intermediate representation (IR) operations and expanding them into micro-operations executed by a semantic SIMT engine.

To demonstrate the capability of the framework, we prototype a warp-level reduction instruction, `warp_reduce_add`, and integrate it into the CUDA programming workflow using inline PTX assembly. The instruction is compiled by Clang into PTX and executed on ProtoGPU through a CUDA Runtime shim path. Current repository evidence validates the instruction semantics on focused demos and unit/integration tests, and the quantitative results reported in this paper are grounded in reproducible experiment logs from that workflow.

ProtoGPU provides a lightweight infrastructure for GPU ISA research and hardware–software co-design, enabling rapid exploration of instruction semantics and compiler integration prior to hardware implementation.

1 Introduction

Modern GPUs continue to evolve with increasingly complex instruction sets to support emerging workloads such as machine learning, graph analytics, and large-scale data processing. Designing and evaluating new GPU instructions is challenging due to several factors:

- GPU architectures are complex and largely proprietary.
- Hardware implementation cycles are long and costly.
- Existing simulation frameworks focus primarily on microarchitectural performance modeling rather than instruction set exploration.

Architecture simulators such as GPGPU-Sim [1] and Accel-Sim [2] provide detailed modeling of GPU hardware pipelines and memory systems. However, these simulators are primarily designed for performance analysis rather than rapid instruction set prototyping. Integrating new instructions into such frameworks often requires significant modifications to the simulated hardware pipeline.

For early-stage ISA exploration, researchers often require a lightweight environment that can:

1. Execute representative GPU programs/demos
2. Rapidly integrate new instructions
3. Validate instruction semantics
4. Evaluate compiler integration

To address these needs, we present **ProtoGPU**, a semantic GPU execution framework designed for **GPU ISA prototyping**.

ProtoGPU executes CUDA and PTX programs using a functional SIMT execution model. Instead of simulating hardware timing behavior, ProtoGPU focuses on instruction semantics and program correctness. New instructions can be rapidly integrated by mapping them to IR operations that are internally expanded into micro-operations executed by the SIMT engine.

To demonstrate the usefulness of ProtoGPU, we implement a new warp-level instruction called `warp_reduce_add`, which performs reduction across threads within a warp. The instruction is exposed to CUDA programs through inline PTX assembly and compiled using Clang. ProtoGPU executes the generated PTX in the current demo workflow and validates the intended semantics in focused test coverage.

This work makes the following contributions:

1. **ProtoGPU**, a semantic GPU execution framework capable of executing CUDA and PTX programs.
2. An **IR-based instruction prototyping mechanism** that enables rapid integration of new GPU instructions.
3. A **CUDA-compatible execution environment** (CUDA Runtime shim + PTX override workflow) that allows evaluation of ISA extensions on executable CUDA demos.
4. A **case study implementing a warp-level reduction instruction**, demonstrating the effectiveness of the framework for ISA exploration.

2 Background

2.1 GPU Execution Model

GPUs use a **SIMT (Single Instruction Multiple Threads)** execution model in which groups of threads called **warps** execute the same instruction simultaneously.

Key concepts include:

- **Warp**: typically 32 threads executing in lock-step
- **Thread blocks**: groups of threads scheduled on streaming multiprocessors
- **PTX**: NVIDIA's intermediate GPU assembly language [4]
- **Warp-level primitives**: operations that allow threads within a warp to communicate efficiently

Warp-level operations such as shuffle and reduction are commonly used in GPU kernels.

2.2 Challenges in GPU ISA Prototyping

Evaluating new GPU instructions typically requires one of the following approaches:

2.2.1 Hardware Implementation

Direct implementation in hardware provides accurate results but requires substantial engineering effort and long design cycles.

2.2.2 Microarchitectural Simulation

Cycle-accurate GPU simulators provide detailed performance modeling but are often complex and difficult to modify.

2.2.3 Dynamic Compilation Frameworks

Some frameworks translate GPU programs to other architectures but do not directly support instruction set exploration.

ProtoGPU aims to provide a lightweight alternative that focuses on **instruction semantics and integration feasibility**.

3 ProtoGPU Architecture

ProtoGPU implements a semantic GPU execution framework capable of running CUDA demo programs through a PTX execution pipeline.

3.1 System Overview

The execution pipeline of ProtoGPU is shown in Fig. 1.

```
CUDA Program
↓
Clang / CUDA Compiler
↓
PTX Code
↓
PTX Parser
↓
Intermediate Representation (IR)
↓
Micro-Operation Expansion
↓
SIMT Execution Engine
```

Figure 1: ProtoGPU execution pipeline from CUDA source to semantic SIMT execution.

ProtoGPU operates at the **semantic execution level**, focusing on functional correctness rather than microarchitectural timing.

3.2 CUDA Integration

ProtoGPU includes a lightweight **CUDA compatibility shim** that allows selected CUDA demo kernels to execute within the framework.

The validated workflow is as follows:

1. CUDA programs are compiled using Clang.
2. Inline PTX assembly allows custom instructions to be embedded.
3. PTX text is provided to the shim (for example via PTX override) and executed by ProtoGPU.

This mechanism allows researchers to evaluate ISA extensions using executable CUDA demos in a functional (non-cycle-accurate) setting.

3.3 IR-Based Instruction Expansion

A key feature of ProtoGPU is its **IR-based instruction prototyping mechanism**.

Each PTX instruction is mapped to one or more **IR operations**. These IR operations are further expanded into **micro-operations (uops)** executed by the SIMT engine.

Example mapping is shown in Fig. 2.

```
PTX Instruction
      ↓
  IR Operation
      ↓
Micro-operations
```

Figure 2: Instruction mapping flow from PTX instruction to IR operation and micro-operations.

This design allows new instructions to be rapidly integrated by defining IR mappings and micro-operation expansions.

4 Instruction Prototyping Mechanism

Integrating a new instruction into ProtoGPU follows a three-step workflow.

4.1 Step 1: PTX-ISA Mapping

Add the new opcode form to the PTX-ISA mapping asset so frontend decoding can map it to an IR opcode.

Example:

```
warp_reduce_add
```

4.2 Step 2: Descriptor Expansion

Add an instruction descriptor entry that expands the IR opcode into one or more micro-operations.

4.3 Step 3: Micro-Operation Semantics

Implement (or reuse) execution semantics for the new micro-operation path in the SIMT execution engine.

This approach allows rapid experimentation with new instruction semantics.

5 Case Study: warp_reduce_add

To demonstrate the capability of ProtoGPU, we prototype a warp-level reduction instruction called `warp_reduce_add`.

5.1 Motivation

Warp-level reductions are common in GPU kernels, such as summation and histogram computations.

Traditional implementations require multiple instructions:

```
shuffle
add
loop
```

This sequence introduces additional instruction overhead.

5.2 Proposed Instruction

We propose the instruction:

```
warp_reduce_add
```

Semantics:

```
r0 = sum of values across threads in the warp
```

The instruction performs a warp-level reduction and returns the result to each participating thread.

5.3 CUDA Integration

The instruction is exposed to CUDA programs using inline PTX [3,4]:

```
asm volatile("warp_reduce_add.f32 %0, %1;" : "=f"(out) : "f"(x));
```

Clang compiles this code and generates PTX; support for custom instructions in this workflow is validated through ProtoGPU's execution pipeline and test artifacts [6,10].

ProtoGPU successfully parses and executes the instruction.

5.4 Implementation in ProtoGPU

The instruction integration flow is shown in Fig. 3.

```
warp_reduce_add
      ↓
IR opcode:  warp_reduce_add
      ↓
EXEC uop:  WARP_REDUCE_ADD
```

Figure 3: Integration path of `warp_reduce_add` from instruction form to IR opcode and execution micro-operation.

The IR operation coordinates data exchange and accumulation across threads within a warp.

6 Evaluation

We evaluate ProtoGPU using focused warp-reduction demos and tests in the current repository state.

6.1 Experimental Setup

- CUDA kernels compiled using Clang
- PTX executed on ProtoGPU through the CUDA Runtime shim path
- Comparison target: custom `warp_reduce_add` path vs shuffle-based reduction PTX path

All measurements in Section 6.2 were collected on Linux from the public ProtoGPU `v0.1.0` release [10], using the repository shim integration scripts and shim-exported trace/stats artifacts. The evaluated PTX targets `.version 6.4` and `.target sm_70`, and the custom-op workflow uses a split host-binary/PTX-override path so that Clang-generated PTX can be executed through the CUDA Runtime shim. These measurements report functional-semantic execution counters rather than cycle-level hardware events.

6.2 Instruction Count Reduction

We report two complementary metrics from shim-side dynamic stats:

1. **Global dynamic count** (end-to-end kernel-level): `inst.fetch`
2. **Semantics-local equivalent count** (reduction idiom only): baseline `shfl + add` sequence counters versus `warp_reduce_add`

In this experiment, `inst.commit` matched `inst.fetch` in both paths, providing a consistency check for global dynamic instruction counts.

6.2.1 Global dynamic count (end-to-end)

Baseline is measured using `inst.fetch`. Evidence artifacts are available in `build/test_out/baseline.stats.json` and `build/test_out/custom.stats.json`.

Table 1: Global dynamic count (end-to-end).

Workload	Baseline	Custom	Delta (%)
`warp_reduce_add_demo` kernel	41	24	-41.5

6.2.2 Semantics-local equivalent count (reduction idiom only)

For the reduction idiom itself, we compare only semantically equivalent operations:

- Baseline equivalent sequence: `uop.exec.shfl.down` + `uop.exec.shfl.idx` + `uop.exec.add.f32` = 3 + 1 + 3 = 7
- Custom instruction path: `uop.exec.warp_reduce_add.f32` = 1

This yields an **85.7% reduction** for the reduction operation sequence itself.

Table 2: Semantics-local equivalent dynamic count.

Metric scope	Baseline ops	Custom ops	Delta (%)
Reduction idiom only	7	1	-85.7

Artifacts are available in `build/test_out/baseline.stats.json` and `build/test_out/custom.stats.json`.

6.2.3 Static PTX instruction-line count (kernel body)

From generated PTX files, we also report static kernel-body instruction-line counts as a supplementary metric.

Table 3: Static PTX instruction-line count (kernel body).

Workload	Baseline	Custom	Delta (%)
`warp_reduce_add_demo` kernel	40	23	-42.5

Static PTX artifacts are available in `build/test_out/warp_reduce_add_demo_alternative.ptx` and `build/test_out/warp_reduce_add_demo.ptx`.

Note: the static whole-kernel PTX counting rule counts instruction lines with explicit opcode + operand formatting and does not include `ret;`. Therefore, static totals are one instruction lower than dynamic `inst.fetch` totals for both paths; this does not affect the relative reduction trend.

6.2.4 Static PTX semantics-local equivalent count

To exclude unrelated kernel instructions, we also compare only the static PTX instructions implementing the reduction idiom:

- Baseline equivalent PTX sequence: `shfl.sync.*` + `add.rn.f32` = 4 + 3 = 7
- Custom PTX instruction: `warp_reduce_add.f32` = 1

Table 4: Semantics-local equivalent static PTX count.

Metric scope	Baseline ops	Custom ops	Delta (%)
Reduction idiom only (static PTX)	7	1	-85.7

This yields an **85.7% static reduction** for the reduction idiom itself.

Static PTX artifacts are available in `build/test_out/warp_reduce_add_demo_alternative.ptx` and `build/test_out/warp_reduce_add_demo.ptx`.

6.2.5 Reproducible measurement procedure

1. Run the custom instruction path with shim stats export:

```
GPUSIM_TRACE=$PWD/build/test_out/custom.trace.jsonl \
GPUSIM_STATS=$PWD/build/test_out/custom.stats.json \
bash scripts/run_cuda_shim_e2e_warp_reduce_add_demo_cu.sh build
```

2. Run the shuffle-based baseline path with shim stats export:

```
GPUSIM_TRACE=$PWD/build/test_out/baseline.trace.jsonl \
GPUSIM_STATS=$PWD/build/test_out/baseline.stats.json \
bash scripts/run_cuda_shim_e2e_warp_reduce_add_demo_alternative_cu.sh build
```

3. Read `inst.fetch` and `inst.commit` from the generated stats JSON files for the global end-to-end metric.

4. Read `uop.exec.shfl.down`, `uop.exec.shfl.idx`, `uop.exec.add.f32`, and `uop.exec.warp_reduce_add.f32` for the semantics-local equivalent metric.

5. Report both metrics: global dynamic count and semantics-local equivalent count.

6. For static PTX kernel-body counting, count instruction lines inside the kernel entry body in `build/test_out/warp_reduce_add_demo_alternative.ptx` and `build/test_out/warp_reduce_add_demo.ptx`.

7. For static PTX semantics-local counting, count only `shfl.sync.*` and `add.rn.f32` in the baseline PTX kernel body, and `warp_reduce_add.f32` in the custom PTX kernel body.

6.3 Program Simplification

Using `warp_reduce_add` simplifies kernel implementations in focused warp-reduction demos. Quantitative synchronization/counter reductions should be reported only with attached experiment artifacts.

6.4 Limitations

The evaluation in this paper is intentionally focused on a warp-reduction bring-up path rather than a broad workload suite. As a result, the reported instruction-count reductions should be interpreted as evidence that ProtoGPU can support semantically correct ISA prototyping and focused quantitative comparison, not as a claim of general performance improvement across arbitrary CUDA workloads. Broader external validation would require additional kernels, more instruction classes, and larger benchmark coverage.

7 Related Work

Existing GPU simulation frameworks primarily focus on microarchitectural modeling [1,2,8,9].

7.1 GPU Architecture Simulators

Cycle-accurate GPU simulators provide detailed performance analysis but are complex to modify for ISA prototyping [1,2,8,9].

7.2 Dynamic PTX Frameworks

Dynamic compilation frameworks translate PTX programs to alternative execution environments [7].

ProtoGPU differs from these systems by focusing on **rapid instruction integration and semantic validation**.

8 Conclusion

This paper presented **ProtoGPU**, a semantic GPU execution framework for rapid prototyping of GPU ISA extensions.

ProtoGPU enables researchers to integrate new instructions, execute CUDA demos through the shim workflow, and validate instruction semantics without requiring cycle-accurate hardware simulation.

Through a focused case study implementing the `warp_reduce_add` instruction, we show that ProtoGPU can support rapid ISA bring-up, compiler-adjacent integration, and semantic validation in a functional SIMT execution setting.

Future work includes extending the framework to support additional ISA extensions, broadening workload coverage, and exploring compiler-driven instruction generation.

References

[1] A. Bakhoda, G. L. Yuan, W. W. L. Fung, H. Wong, and T. M. Aamodt, "Analyzing CUDA Workloads Using a Detailed GPU Simulator," in *2009 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2009.

[2] M. Khairy, J. Shen, T. M. Aamodt, and T. G. Rogers, "Accel-Sim: An Extensible Simulation Framework for Validated GPU Modeling," in *Proceedings of the 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020.

[3] NVIDIA Corporation, "CUDA C++ Programming Guide (CUDA 10.1)," `https://docs.nvidia.com/cuda/archive/10.1/cuda-c-programming-guide/`, accessed 2026-04-18.

[4] NVIDIA Corporation, "Parallel Thread Execution ISA Version 6.4," `https://docs.nvidia.com/cuda/archive/10.1/pdf/ptx_isa_6.4.pdf`, accessed 2026-04-18.

[5] C. Lattner and V. Adve, "LLVM: A Compilation Framework for Lifelong Program Analysis and Transformation," in *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*, 2004.

[6] LLVM Project, "Compiling CUDA with Clang (LLVM CUDA Support)," `https://llvm.org/docs/CompileCudaWithLLVM.html`, accessed 2026-04-18.

[7] G. Diamos, A. Kerr, S. Yalamanchili, and N. Clark, "Ocelot: A Dynamic Optimization Framework for Bulk-Synchronous Applications in Heterogeneous Systems," in *Proceedings of the 19th International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2010.

[8] J. Power, J. Hestness, M. S. Orr, M. D. Hill, and D. A. Wood, "gem5-gpu: A Heterogeneous CPU-GPU Simulator," *IEEE Computer Architecture Letters*, vol. 13, no. 1, pp. 34–36, 2014.

[9] R. Ubal, J. Sahuquillo, S. Petit, and P. Lopez, "Multi2Sim: A Simulation Framework to Evaluate Multicore-Multithreaded Processors," in *Proceedings of the 16th International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*, 2007.

[10] H. Zhenzhong, "ProtoGPU v0.1.0," GitHub, Apr. 2026. [Online]. Available: `https://github.com/Han-Zhenzhong/ProtoGPU/releases/tag/v0.1.0`